

MASTER PROMPT — AI-Powered Criminal Network Analysis Platform (Smart India Hackathon)

RUN MODE: 100% LOCAL, NO DOCKER. Everything must run with only Python 3.11+, Node.js 18+, and npm installed. No containers, no external database servers, no cloud services.

You are building a complete, production-quality web application end to end. Work through the phases below IN ORDER. At the end of each phase, run the app, verify the phase's acceptance criteria in the browser, fix all errors, and only then move to the next phase. Do not skip ahead. Maintain a clean monorepo and commit logically per feature.

=====

PROJECT OVERVIEW

=====

An AI-assisted investigation platform for law enforcement. It ingests multi-source case data (CDRs, financial transactions, FIRs, surveillance reports, social media intel, criminal-history records), extracts entities (people, phone numbers, vehicles, organisations, locations), builds a relationship graph via a bounded, investigator-gated depth-first traversal, scores each node's suspicion level, detects 12 suspicious activity patterns, and presents everything as interactive visual intelligence: network graph with concentric confidence zones, hierarchy tree, past-event timeline, predictive future timeline, and a live location-tracking map (India-scoped).

Three non-negotiable design principles:

1. DYNAMIC ON-DEMAND INGESTION — data is fetched only for authorised individuals in an active case, never bulk-ingested for a whole population.
2. INVESTIGATOR-GATED EXPANSION — the graph grows one hop at a time, only after investigator approval (or a standing case-level authorisation).
3. BOUNDED EXPLORATION — traversal into a branch stops when a node's estimated network-membership probability drops below 25%.

=====

SYSTEM ARCHITECTURE (seven cooperating layers)

=====

1. Data Ingestion Layer — mock pluggable per-person connectors (CDR, transactions, FIR, surveillance, social media, criminal history) served from local synthetic files; invoked ONLY for an already-authorised individual.
2. Entity & Relationship Extraction Layer — structured ETL parsers for tabular sources; NLP/NER pipeline for unstructured text; entity resolution/dedup.
3. Graph Engine — maintains the evolving case graph, runs the bounded DFS traversal, stores per-node suspicion scores and per-edge evidence types.
4. Analytics & Pattern Detection Engine — 12 detectors, hierarchy/influence scoring, predictive future-timeline forecasting.
5. Access Control & Authorisation Layer — per-investigator, per-case, per-node RBAC; elevated approval gate for location tracking; full audit logging.
6. Visualisation / Frontend Layer — interactive graph, hierarchy tree, timelines, map.
7. Investigator Feedback Loop — manual overrides (drag-to-reclassify, pattern confirm/reject, hierarchy re-rank) feed back into scoring weights.

Data flow: Ingestion → Extraction → Graph Engine (traversal + scoring) → Analytics → Visualisation, with the Authorisation Layer gating every hop between Ingestion and the Graph Engine, and the Feedback Loop flowing from the UI back into scoring.

=====

TECHNOLOGY STACK (NO DOCKER — use exactly this)

=====

- Frontend: React (Vite) + Cytoscape.js (network graph), D3.js (timelines), Leaflet with OpenStreetMap tiles bounded to India (location tracking). Run: ``npm run dev``.
- Backend: Python FastAPI with REST + WebSocket endpoints, run via ``uvicorn app.main:app --reload`` inside a virtualenv.

- Graph storage: a GraphStore ABSTRACTION LAYER backed by NetworkX + SQLite persistence as the PRIMARY store (no server needed). Keep an optional Neo4j adapter behind the same interface for anyone who has Neo4j installed — but the app must run fully without it.
- Raw record storage: SQLite (via SQLAlchemy) for CDRs, transactions, FIR text, case metadata, users, audit logs. Single file DB, zero setup.
- NLP/NER: spaCy (en_core_web_sm) with custom EntityRuler patterns for Indian phone numbers, vehicle registrations (e.g. MH12AB1234), FIR numbers, and names. Architect so a fine-tuned transformer can be swapped in later.
- Analytics/ML: NetworkX (centrality, Louvain community detection, cycle detection), scikit-learn (IsolationForest, DBSCAN), statsmodels or simple exponential smoothing for forecasting.
- Entity resolution: rapidfuzz fuzzy matching.
- Auth: OAuth2 password flow + JWT, roles: investigator, senior_authority, admin; per-case and per-node scoping; full audit logging.
- Dev experience: a root-level `run.sh` / `run.bat` (and README instructions) that
 - (a) creates the Python venv + installs requirements + downloads the spaCy model,
 - (b) installs frontend deps, (c) starts backend on :8000 and frontend on :5173 concurrently. Also provide `make dev` for Unix users. NO docker-compose anywhere.
- Config: all thresholds (25% pruning, 75% membership, zone bands, scoring weights) in a single config.yaml — never hardcoded.

Repo layout:

/frontend, /backend, /data-generator, /config, run.sh, run.bat, README.md

=====

DATA MODEL / GRAPH SCHEMA

=====

Node types (all carry: id, created_at, source_refs[]):

- Person: name, photo_url, phone_numbers[], criminal_history_flag, suspicion_score (0–1), hierarchy_score, confidence_band (>75 / 50–75 / 25–50 / <25), explored (bool)
- Location: lat, lng, address, associated_event_ids[]
- Organisation: name, type, associated_person_ids[]
- Object/Vehicle: registration_number, type, associated_person_ids[], event_ids[]
- Event: timestamp, type (crime_event | network_event | predicted_event), description, linked_entity_ids[], is_predicted (bool)

Edge types (all carry: confidence 0–1, evidence_type tag, source record reference):

- CALL (timestamp, duration, frequency), TRANSACTION (amount, timestamp, direction), FIR (case_ref, date, offence), SURVEILLANCE (source, timestamp, description), SOCIAL_MEDIA (platform, interaction_type), CRIMINAL_HISTORY (shared case), CO_LOCATION (derived — computed, never sourced from a single record).

=====

CORE ALGORITHM — BOUNDED, INVESTIGATOR-GATED DFS TRAVERSAL

=====

Implement exactly this flow as a backend service with WebSocket progress events:

1. Investigator names a SEED person and uploads/authorises that person's dataset.
2. Extraction layer parses ONLY that person's data → entities + typed edges.
3. Nodes/edges are pushed to the frontend graph in real time (WebSocket).
4. Every entity linked to the seed becomes a CANDIDATE for the next hop.
5. Each candidate gets a provisional suspicion score (formula below).
6. AUTHORISATION GATE: if score ≥ 0.25 , emit an approval request to the investigator UI (approve/reject buttons per candidate). If the case has "standing authorisation" enabled, expansion proceeds automatically.
7. On approval, fetch that candidate's data from its mock connector and recurse DEPTH-FIRST (one branch fully before siblings).
8. PRUNING: if score < 0.25 , mark the node unexplored — it still renders on the graph

as a low-confidence '?' placeholder (face hidden), but no data is ever fetched.

9. CONVERGENCE: stop when no unexplored candidate ≥ 0.25 remains.

The graph at any moment must reflect exactly the data fetched so far — nothing pre-emptive. Log every fetch and every authorisation decision to the audit log.

SUSPICION SCORE (recomputed as data arrives; weighted combination, weights in config.yaml):

- strength/frequency of links to confirmed ($>75\%$) members
- recency of those links (exponential decay)
- corroboration across independent evidence types (multi-source links weigh more)
- network centrality (degree + betweenness)
- prior criminal-history record (boolean boost)
- investigator manual override adjustments (from the feedback loop)

Map score $\rightarrow$ bands: >0.75 inner zone (member by default), $0.50\text{--}0.75$ middle ring, $0.25\text{--}0.50$ outer ring, <0.25 outside all rings, unexplored.

=====

SUSPICIOUS PATTERN DETECTION ENGINE — 12 DETECTORS

=====

Each is an independent, unit-tested detector class with a common interface

(`run(case_graph) -> list[PatternAlert]`); alerts appear in a UI panel and on the timeline:

1. Sudden communication burst — time-series anomaly detection on per-pair call frequency.
2. New connections between unrelated people — diff between successive graph snapshots.
3. One person bridging multiple groups — betweenness centrality / bridge detection.
4. Same entity across multiple cases — cross-case entity matching.
5. Unusual location sequence — sequence modelling on geo-stamped events; flag improbable transitions (speed/distance sanity checks).
6. Repeated co-location of multiple people — spatiotemporal clustering (DBSCAN on

lat/lng/time).

7. Unusual financial transaction pattern — IsolationForest outliers on amount/frequency/counterparties.
8. Circular transaction pattern — cycle detection on the transaction sub-graph.
9. Communication + location correlation — joint analysis of call timing vs co-location.
10. Phone/vehicle/identity hopping — entity-resolution engine flags rapid device/alias switching (rapidfuzz + temporal analysis).
11. Sudden formation of a new community — Louvain community detection across snapshots.
12. Existing network suddenly more active — trend analysis on aggregate edge activity within a known cluster.

ENTITY RESOLUTION: fuzzy-matching/deduplication (rapidfuzz) merging same-person references across spellings, aliases, and identifiers; this same engine powers detector 10.

=====

UI / UX SPECIFICATION (follow strictly)

=====

Colour palette — ONLY four colours: white (dominant), yellow (secondary accent), red (reserved strictly for high-importance: crime events, high alerts), black (borders, text, timeline base line).

8.1 Interactive Network Graph (main view):

- People render as circular photo frames, name beneath; criminal-history individuals get a small badge icon on the frame. Never plain-text labels.
- All relationships render as connecting lines; evidence-type text labels appear next to edges ONLY when zoomed past a threshold (uncluttered default view).
- THREE CONCENTRIC CONFIDENCE ZONES behind the network: innermost >75%, middle 50–75%, outer 25–50%; <25% nodes sit outside all rings with a generic '?' placeholder face. Zone radius scales with node count per zone.

- Click a person → side panel: FIR/criminal history, connected people, associated locations/organisations/objects, hierarchy position, exact numeric score (scores are spatial in the main view, numeric only in the panel).
- Click an edge → side panel: every evidence type as icon+label; clicking an icon drills into the underlying record.
- Drag-to-reclassify: investigator drags a node between zones; the override is logged and fed back into the scoring model (feedback loop).
- Ordinary relationships render even when one side is outside the inner zone (e.g. a suspect's mother).
- A person can appear via physical evidence alone (ID card / vehicle at a crime scene) before any person-to-person link exists.
- Below the graph: three pivot filters — Location, Organisation, Object/Vehicle — that re-centre the graph around the chosen entity and show all tied people.

8.2 Hierarchy Tree: same nodes, arranged by influence score (PageRank + betweenness); manual re-ranking allowed and logged; same visual style; clicking a node jumps to it on the main graph.

8.3 Timeline (past): single horizontal line; white dots = general network events, red dots = crime events; click opens details.

8.4 Predictive Future Timeline: same style, forward-looking; populated by forecasts from communication/movement/transaction trends; clicking a predicted event shows the model's what/where/how estimate with an explicit uncertainty note ("one input among others, not a certainty").

8.5 Investigator Input: investigators define custom suspicious-activity rules that run ALONGSIDE (not replacing) the built-in 12 detectors.

8.6 Dynamic Location Tracking: for >75%-zone individuals WITH elevated authorisation only — live Leaflet map bounded to India, WebSocket-fed position updates, movement trails from a simulated local ping stream.

=====

SECURITY, PRIVACY & COMPLIANCE (implement, don't just document)

=====

- RBAC: investigator accounts scoped per-case and per-node.
- Elevated authorisation workflow: continuous location tracking requires a national-security justification field OR explicit sign-off by a senior_authority role account — build the approval request/grant UI flow.
- Audit log: every data fetch, authorisation decision, and manual override, with actor, timestamp, and payload; admin-viewable audit screen.
- TLS-ready config; encrypt sensitive columns at rest (e.g. Fernet/SQLCipher-style column encryption in SQLite).
- Data minimisation enforced structurally by the on-demand traversal itself.

=====

PHASE 0 — SCAFFOLD + SYNTHETIC DATA (do this first)

=====

- Monorepo: /frontend, /backend, /data-generator, /config, run.sh, run.bat, README.
- Backend venv + requirements.txt; frontend Vite scaffold; SQLite auto-created on first run; GraphStore abstraction (NetworkX + SQLite persistence) wired in.
- Build a SYNTHETIC DATA GENERATOR producing a realistic fictional case: ~40 people (initials-based avatar placeholders generated locally), CDR CSVs, transaction CSVs, FIR text documents (realistic Indian FIR format), surveillance notes, social media posts, criminal-history records, and geo-stamped location pings — deliberately seeded so ALL 12 patterns are detectable and the DFS traversal produces an interesting multi-ring graph with a clear kingpin, lieutenants, peripherals, and innocent contacts that fall below 25%.
- Mock "external source connectors" serving this data per-person on demand, simulating the pluggable Ingestion Layer (government interoperability out of scope).

ACCEPTANCE: `run.sh` (or run.bat) starts backend on :8000 and frontend on :5173 with

one command; generator produces the dataset; /health endpoint passes; no Docker anywhere in the repo.

=====

PHASE 1 — MVP

=====

- Auth (login, JWT, roles), case creation.
- Manual/batch entity upload endpoint + parser for the structured synthetic files.
- Static graph rendering with the full three-zone concentric visual design, circular photo frames, badges, zoom-gated edge labels.
- Node and edge detail side panels with evidence icons and record drill-down.
- Basic hierarchy tree view.

ACCEPTANCE: upload the synthetic case → graph renders correctly in zones; panels show correct data; hierarchy tree displays; all in the four-colour palette.

=====

PHASE 2 — CORE INTELLIGENCE

=====

- NLP/NER extraction pipeline for FIRs, surveillance notes, social posts (+ entity resolution/dedup with rapidfuzz).
- The bounded, investigator-gated DFS traversal with live WebSocket graph growth, approval-gate UI (per-candidate approve/reject + standing-authorisation toggle), 25% pruning, '?' placeholder nodes.
- Full suspicion-score engine with recomputation on new data.
- All 12 pattern detectors + alerts panel; each detector unit-tested (pytest) against the seeded synthetic patterns.
- Past-events timeline (white/red dots).

ACCEPTANCE: starting from the seed suspect, the traversal demo runs end to end with approval prompts; all 12 detectors fire on the seeded data; pruned innocents render

as '?' outside the rings; timeline reflects extracted events.

=====

PHASE 3 — ADVANCED FEATURES

=====

- Predictive future timeline with forecasting from the dynamic synthetic streams.
- Dynamic location tracking: elevated-authorisation request/approval workflow (senior_authority role), then live India-bounded map with movement trails from a simulated ping stream.
- Full RBAC enforcement per-case/per-node + complete audit-log UI.
- Investigator feedback loop: drag-to-reclassify, pattern confirm/reject, and hierarchy re-ranking all adjust scoring weights; custom investigator rules engine.
- Incremental graph updates as new dynamic data arrives (no full reprocessing).
- Location/Organisation/Object pivot filters below the graph.

ACCEPTANCE: full end-to-end demo script works: login → create case → seed suspect → gated traversal → alerts fire → hierarchy → timelines → request tracking authorisation → senior approves → live map updates → investigator overrides adjust scores → audit log shows everything.

=====

FINAL DELIVERABLES

=====

- Working app via one local command (run.sh / run.bat / make dev), seeded demo case, step-by-step demo walkthrough in README (including venv + npm setup for a fresh machine with only Python and Node installed).
- Unit tests (pytest) for the traversal algorithm, scoring, and all 12 detectors.
- Architecture diagram + API docs (FastAPI auto-docs at /docs).
- A 5-minute demo storyline document for hackathon judging.

Throughout: prefer clarity over cleverness, keep detectors and connectors pluggable, never hardcode the 25%/75% thresholds (config.yaml), and verify each phase visually in the browser before proceeding.